



1. Problem Title & Link

Title: LeetCode 930 : Binary Subarrays With Sum

Link: [LeetCode 930 - Binary Subarrays With Sum](#)

2. Problem Statement (Short Summary)

You are given a binary array:

nums[]

and an integer:

goal

Return the number of non-empty subarrays whose sum equals:

goal

In simple terms:

Count all continuous subarrays having sum exactly equal to goal.

3. Examples (Input → Output)

Example 1

Input:

nums=[1,0,1,0,1]

goal=2

Output:

4

Explanation:

Valid subarrays:

[1,0,1]

[1,0,1,0]

[0,1,0,1]

[1,0,1]

Example 2

Input:

nums=[0,0,0,0,0]

goal=0



Output:

15

4. Constraints

$1 \leq \text{nums.length} \leq 3 \times 10^4$

$\text{nums}[i] \in \{0,1\}$

$0 \leq \text{goal} \leq \text{nums.length}$

Special restrictions:

- Need count of all subarrays
- Better than $O(n^2)$

5. Core Concept (Pattern / Topic)

Prefix Sum + HashMap

6. Thought Process (Step-by-Step Explanation)

Brute Force

Generate every subarray:

1. Compute sum
2. Compare with goal
3. Increment count

Time:

$O(n^2)$

Too slow.

Optimized Thinking

Observation:

For prefix sums:

$\text{currentSum} - \text{previousSum}$

=

goal

Rearranging:

previousSum

=

$\text{currentSum} - \text{goal}$

If we know how many times:

$\text{currentSum} - \text{goal}$

occurred before, then those many subarrays exist.



Algorithm:

1. Maintain running prefix sum
2. Store frequencies in HashMap
3. For each element:
 - Update sum
 - Search:

sum-goal

- Add frequency to answer
- Store current sum frequency

7. Visual / Intuition Diagram (ASCII Diagram)

Input:

nums=[1,0,1,0,1]

goal=2

Prefix sums:

1,1,2,2,3

Check:

sum-goal

Occurrences:

3-2=1

Prefix:

1 occurred twice

Meaning:

2 subarrays found

8. Pseudocode (Language Independent)

map={0:1}

sum=0

count=0

for num in nums:

sum+=num



```
count +=
map.get(
    sum-goal,
    0
)

map[sum]=
map.get(
    sum,
    0
)+1

return count
```

9. Code Implementation

Python

class Solution:

```
def numSubarraysWithSum(
    self,
    nums,
    goal):

    prefixMap={0:1}

    currentSum=0

    result=0

    for num in nums:

        currentSum+=num

        result += (
            prefixMap.get(
                currentSum-goal,
                0
```



```
    )  
    )  
  
    prefixMap[  
        currentSum  
    ]=(  
        prefixMap.get(  
            currentSum,  
            0  
        )+1  
    )  
}
```

return result

Java

```
class Solution {  
  
    public int numSubarraysWithSum(  
        int[] nums,  
        int goal) {  
  
        Map<Integer,Integer>  
            map=  
            new HashMap<>();  
  
        map.put(0,1);  
  
        int currentSum=0;  
  
        int result=0;  
  
        for(int num:nums){  
  
            currentSum+=num;  
  
            result+=  
                map.getOrDefault(  

```



```
        currentSum-goal,
        0
    );

    map.put(
        currentSum,
        map.getDefault(
            currentSum,
            0
        )+1
    );
}

return result;
}
```

10. Time & Space Complexity

Time Complexity:

$O(n)$

Reason:

Array traversed once.

Space Complexity:

$O(n)$

Reason:

HashMap stores prefix sum frequencies.

11. Common Mistakes / Edge Cases

1. Forgetting initialization:

`map={0:1}`

Without this:

Subarrays beginning from index 0 fail.

2. Using Set instead of HashMap

Wrong:

Set only stores existence



Need frequency count.

3. Updating map before calculation

Wrong order:

Insert first

Calculate later

4. Missing:

goal=0

case

12. Detailed Dry Run (Step-by-Step Table)

Input:

nums=[1,0,1,0,1]

goal=2

Index	Value	Current Sum	Sum-Goal	Count Added	Total
0	1	1	-1	0	0
1	0	1	-1	0	0
2	1	2	0	1	1
3	0	2	0	1	2
4	1	3	1	2	4

Final:

4

13. Common Use Cases (Real-Life / Interview)

Binary event tracking

Network packet analysis

Log sequence counting

Signal pattern matching

14. Common Traps (Important!)

1. Missing:

map={0:1}

2. Using Set instead of HashMap



3. Wrong update order
4. Using nested loops

15. Builds To (Related LeetCode Problems)

Progression:

- LC 930 → Binary Subarrays With Sum
- LC 560 → Subarray Sum Equals K
- LC 974 → Subarrays Divisible By K
- LC 523 → Continuous Subarray Sum
- LC 525 → Contiguous Array

16. Alternate Approaches + Comparison

Approach	Time	Space	Notes
Brute Force	$O(n^2)$	$O(1)$	Too slow
Sliding Window (AtMost trick)	$O(n)$	$O(1)$	Works only for binary arrays
Prefix Sum + HashMap	$O(n)$	$O(n)$	General optimal approach

17. Why This Solution Works (Short Intuition)

If a previous prefix sum equals:

$\text{currentSum} - \text{goal}$

then removing that prefix leaves a subarray whose sum is exactly goal. The HashMap efficiently counts how many such prefixes exist.

18. Variations / Follow-Up Questions

1. Return subarray indices
2. Find longest subarray with given sum
3. Count subarrays with $\text{sum} \leq \text{goal}$
4. Support negative numbers
5. Solve using sliding window for binary arrays
- 6.